

Representing and Mining Text

Fundamental concepts: *The importance of constructing mining-friendly data representations; Representation of text for data mining.*

Exemplary techniques: *Bag of words representation; TFIDF calculation; N-grams; Stemming; Named entity extraction; Topic models.*

Up to this point we've ignored or side-stepped an important stage of the data mining process: data preparation. The world does not always present us with data in the feature vector representation that most data mining methods take as input. Data are represented in ways natural to problems from which they were derived. If we want to apply the many data mining tools that we have at our disposal, we must either engineer the data representation to match the tools, or build new tools to match the data. Top-notch data scientists employ both of these strategies. It generally is simpler to first try to engineer the data to match existing tools, since they are well understood and numerous.

In this chapter, we will focus on one particular sort of data that has become extremely common as the Internet has become a ubiquitous channel of communication: text data. Examining text data allows us to illustrate many real complexities of data engineering, and also helps us to understand better a very important type of data. We will see in [Chapter 14](#) that although in this chapter we focus exclusively on text data, the fundamental principles indeed generalize to other important sorts of data.

We've encountered text once before in this book, in the example involving clustering news stories about Apple Inc. ("[Example: Clustering Business News Stories](#)"). There we deliberately avoided a detailed discussion of how the news stories were prepared because the focus was on clustering, and text preparation would have been too much of a digression. This chapter is devoted to the difficulties and opportunities of dealing with text.

In principle, text is just another form of data, and text processing is just a special case of representation engineering. In reality, dealing with text requires dedicated pre-processing steps and sometimes specific expertise on the part of the data science team.

Entire books and conferences (and companies) are devoted to text mining. In this chapter we can only scratch the surface, to give a basic overview of the techniques and issues involved in typical business applications.

First, let's discuss why text is so important and why it's difficult.

Why Text Is Important

Text is everywhere. Many legacy applications still produce or record text. Medical records, consumer complaint logs, product inquiries, and repair records are still mostly intended as communication between people, not computers, so they're still "coded" as text. Exploiting this vast amount of data requires converting it to a meaningful form.

The Internet may be the home of "new media," but much of it is the same form as old media. It contains a vast amount of text in the form of personal web pages, Twitter feeds, email, Facebook status updates, product descriptions, Reddit comments, blog postings—the list goes on. Underlying the search engines (Google and Bing) that we use everyday are massive amounts of text-oriented data science. Music and video may account for a great deal of traffic volume, but when people communicate with each other on the Internet it is usually via text. Indeed, the thrust of Web 2.0 was about Internet sites allowing users to interact with one another as a community, and to generate much added content of a site. This user-generated content and interaction usually takes the form of text.

In business, understanding customer feedback often requires understanding text. This isn't always the case; admittedly, some important consumer attitudes are represented explicitly as data or can be inferred through behavior, for example via five-star ratings, click-through patterns, conversion rates, and so on. We can also pay to have data collected and quantified through focus groups and online surveys. But in many cases if we want to "listen to the customer" we'll actually have to read what she's written—in product reviews, customer feedback forms, opinion pieces, and email messages.

Why Text Is Difficult

Text is often referred to as "unstructured" data. This refers to the fact that text does not have the sort of structure that we normally expect for data: tables of records with fields having fixed meanings (essentially, collections of feature vectors), as well as links between the tables. Text of course has plenty of structure, but it is *linguistic* structure—intended for human consumption, not for computers.

Words can have varying lengths and text fields can have varying numbers of words. Sometimes word order matters, sometimes not.

As data, text is relatively *dirty*. People write ungrammatically, they misspell words, they run words together, they abbreviate unpredictably, and punctuate randomly. Even when

text is flawlessly expressed it may contain synonyms (multiple words with the same meaning) and homographs (one spelling shared among multiple words with different meanings). Terminology and abbreviations in one domain might be meaningless in another domain—we shouldn't expect that medical recordkeeping and computer repair records would share terms in common, and in the worst case they would conflict.

Because text is intended for communication between people, *context* is important, much more so than with other forms of data. Consider this movie review excerpt:

“The first part of this movie is far better than the second. The acting is poor and it gets out-of-control by the end, with the violence overdone and an incredible ending, but it's still fun to watch.”

Consider whether the overall sentiment is for or against the film. Is the word *incredible* positive or negative? It is difficult to evaluate any particular word or phrase here without taking into account the entire context.

For these reasons, text must undergo a good amount of preprocessing before it can be used as input to a data mining algorithm. Usually the more complex the featurization, the more aspects of the text problem can be included. This chapter can only describe some of the basic methods involved in preparing text for data mining. The next few subsections describe these steps.

Representation

Having discussed how difficult text can be, let's go through the basic steps to transform a body of text into a set of data that can be fed into a data mining algorithm. The general strategy in text mining is to use the simplest (least expensive) technique that works. Nevertheless, these ideas are the key technology underlying much of web search, like Google and Bing. A later example will demonstrate basic query retrieval.

First, some basic terminology. Most of this is borrowed from the field of Information Retrieval (IR). A *document* is one piece of text, no matter how large or small. A document could be a single sentence or a 100 page report, or anything in between, such as a YouTube comment or a blog posting. Typically, all the text of a document is considered together and is retrieved as a single item when matched or categorized. A document is composed of individual *tokens* or *terms*. For now, think of a token or term as just a word; as we go on we'll show how they can be different from what are customarily thought of as words. A collection of documents is called a *corpus*.¹

1. Latin for “body.” The plural is *corpora*.

Bag of Words

It is important to keep in mind the purpose of the text representation task. In essence, we are taking a set of documents—each of which is a relatively free-form sequence of words—and turning it into our familiar feature-vector form. Each document is one instance but we don't know in advance what the features will be.

The approach we introduce first is called “bag of words.” As the name implies, the approach is to treat every document as just a collection of individual words. This approach ignores grammar, word order, sentence structure, and (usually) punctuation. It treats every word in a document as a potentially important keyword of the document. The representation is straightforward and inexpensive to generate, and tends to work well for many tasks.



Note: Sets and bags

The terms *set* and *bag* have specific meanings in mathematics, neither of which we exactly mean here. A set allows only one instance of each item, whereas we want to take into account the number of occurrences of words. In mathematics a *bag* is a *multiset*, where members are allowed to appear more than once. The bag-of-words representation initially treats documents as bags—multisets—of words, thereby ignoring word order and other linguistic structure. However, the representation used for mining the text often is more complex than just counting the number of occurrences, as we will describe.

So if every word is a possible feature, what will be the feature's value in a given document? There are several approaches to this. In the most basic approach, each word is a token, and each document is represented by a one (if the token is present in the document) or a zero (the token is not present in the document). This approach simply reduces a document to the set of words contained in it.

Term Frequency

The next step up is to use the word count (frequency) in the document instead of just a zero or one. This allows us to differentiate between how many times a word is used; in some applications, the importance of a term in a document should increase with the number of times that term occurs. This is called the *term frequency* representation. Consider the three very simple sentences (documents) shown in Table 10-1.

Table 10-1. Three simple documents.

- d1 jazz music has a swing rhythm
- d2 swing is hard to explain
- d3 swing rhythm is a natural rhythm

Each sentence is considered a separate document. A simple bag-of-words approach using term frequency would produce a table of term counts shown in Table 10-2.

Table 10-2. Term count representation.

	a	explain	hard	has	is	jazz	music	natural	rhythm	swing	to
d1	1	0	0	1	0	1	1	0	1	1	0
d2	0	1	1	0	1	0	0	0	0	1	1
d3	1	0	0	0	1	0	0	1	2	1	0

Usually some basic processing is performed on the words before putting them into the table. Consider this more complex sample document:

Microsoft Corp and Skype Global today announced that they have entered into a definitive agreement under which Microsoft will acquire Skype, the leading Internet communications company, for \$8.5 billion in cash from the investor group led by Silver Lake. The agreement has been approved by the boards of directors of both Microsoft and Skype.

Table 10-3 shows a reduction of this document to a term frequency representation.

Table 10-3. Terms after normalization and stemming, ordered by frequency

Term	Count	Term	Count	Term	Count	Term	Count
skype	3	microsoft	3	agreement	2	global	1
approv	1	announc	1	acquir	1	lead	1
definit	1	lake	1	communic	1	internet	1
board	1	led	1	director	1	corp	1
compani	1	investor	1	silver	1	billion	1

To create this table from the sample document, the following steps have been performed:

- First, the case has been **normalized**: every term is in lowercase. This is so that words like Skype and SKYPE are counted as the same thing. Case variations are so common (consider iPhone, iphone, and IPHONE) that case normalization is usually necessary.
- Second, many words have been **stemmed**: their suffixes removed, so that verbs like *announces*, *announced* and *announcing* are all reduced to the term *announc*. Similarly, stemming transforms noun plurals to the singular forms, which is why *directors* in the text becomes *director* in the term list.
- Finally, **stopwords** have been **removed**. A stopword is a very common word in English (or whatever language is being parsed). The words *the*, *and*, *of*, and *on* are considered stopwords in English so they are typically removed.

Note that the “\$8.5” in the story has been discarded entirely. Should it have been? Numbers are commonly regarded as unimportant details for text processing, but the

purpose of the representation should decide this. You can imagine contexts where terms like “4TB” and “1Q13” would be meaningless, and others where they could be critical modifiers.



Note: Careless Stopword Elimination

A word of caution: stopwords elimination is not always a good idea. In titles, for example, common words may be very significant. For example, *The Road*, Cormac McCarthy’s story of a father and son surviving in a post-apocalyptic world, is very different from John Kerouac’s famous novel *On the Road*— though careless stopwords removal may cause them to be represented identically. Similarly, the recent movie thriller *Stoker* should not be confused with the 1935 film comedy *The Stoker*.²

Table 10-3 shows raw counts of terms. Instead of raw counts, some systems perform a step of normalizing the term frequencies with respect to document length. The purpose of term frequency is to represent the relevance of a term to a document. Long documents usually will have more words—and thus more word occurrences—than shorter ones. This doesn’t mean that the longer document is necessarily more important or relevant than the shorter one. In order to adjust for document length, the raw term frequencies are normalized in some way, such as by dividing each by the total number of words in the document.

Measuring Sparseness: Inverse Document Frequency

So term frequency measures how prevalent a term is in a single document. We may also care, when deciding the weight of a term, how common it is in the entire corpus we’re mining. There are two opposing considerations.

First, a term should not be too rare. For example, say the unusual word *prehensile* occurs in only one document in your corpus. Is it an important term? This may depend on the application. For retrieval, the term may be important since a user may be looking for that exact word. For clustering, there is no point keeping a term that occurs only once: it will never be the basis of a meaningful cluster. For this reason, text processing systems usually impose a small (arbitrary) lower limit on the number of documents in which a term must occur.

Another, opposite consideration is that a term should not be too common. A term occurring in every document isn’t useful for classification (it doesn’t distinguish anything) and it can’t serve as the basis for a cluster (the entire corpus would cluster together).

2. Both of these examples appeared in recent search results on the film review site of a popular search engine. Not everyone is careful with stopwords elimination.

Overly common terms are typically eliminated. One way to do this is to impose an arbitrary upper limit on the number (or fraction) of documents in which a word may occur.

In addition to imposing upper and lower limits on term frequency, many systems take into account the distribution of the term over a corpus as well. The fewer documents in which a term occurs, the more significant it likely is to be to the documents it does occur in. This sparseness of a term t is measured commonly by an equation called *inverse document frequency* (IDF), which is shown in Equation 10-1.

Equation 10-1. Inverse Document Frequency (IDF) of a term

$$\text{IDF}(t) = 1 + \log \left(\frac{\text{Total number of documents}}{\text{Number of documents containing } t} \right)$$

IDF may be thought of as the boost a term gets for being rare. Figure 10-1 shows a graph of $\text{IDF}(t)$ as a function of the number of documents in which t occurs, in a corpus of 100 documents. As you can see, when a term is very rare (far left) the IDF is quite high. It decreases quickly as t becomes more common in documents, and asymptotes at 1.0. Most stopwords, due to their prevalence, will have an IDF near one.

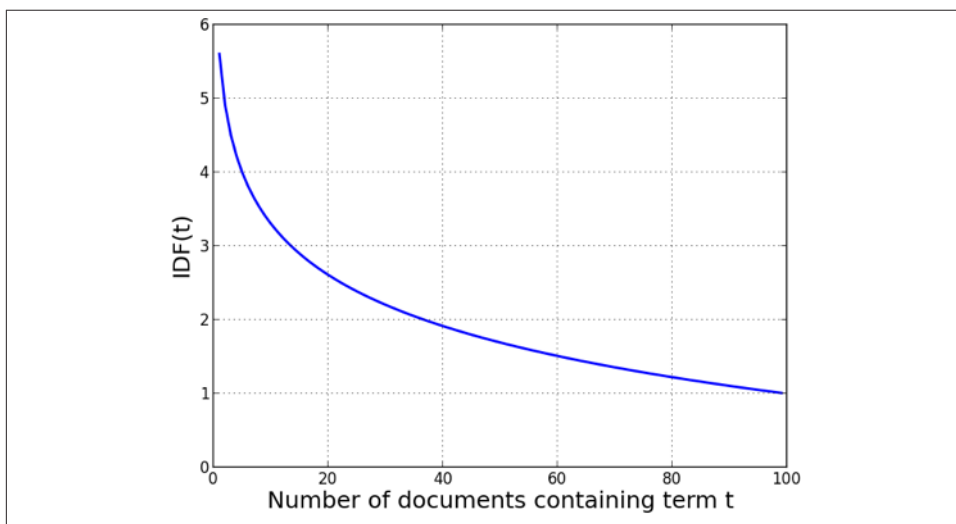


Figure 10-1. IDF of a term t within a corpus of 100 documents.

Combining Them: TFIDF

A very popular representation for text is the **product** of Term Frequency (TF) and Inverse Document Frequency (IDF), commonly referred to as TFIDF. The TFIDF value of a term t in a given document d is thus:

$$\text{TFIDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

Note that the TFIDF value is specific to a single document (d) whereas IDF depends on the entire corpus. Systems employing the bag-of-words representation typically go through steps of stemming and stopword elimination before doing term counts. Term counts within the documents form the TF values for each term, and the document counts across the corpus form the IDF values.

Each document thus becomes a feature vector, and the corpus is the set of these feature vectors. This set can then be used in a data mining algorithm for classification, clustering, or retrieval.

Because there are very many potential terms with text representation, feature selection is often employed. Systems do this in various ways, such as imposing minimum and maximum thresholds of term counts, and/or using a measure such as information gain³ to rank the terms by importance so that low-gain terms can be culled.

The bag-of-words text representation approach treats every word in a document as an independent potential keyword (feature) of the document, then assigns values to each document based on frequency and rarity. TFIDF is a very common value representation for terms, but it is not necessarily optimal. If someone describes mining a text corpus using bag of words it just means they're treating each word individually as a feature. Their values could be binary, term frequency, or TFIDF, with normalization or without. Data scientists develop intuitions about how best to attack a given text problem, but they'll typically experiment with different representations to see which produces the best results.

Example: Jazz Musicians



Having **introduced** a few basic concepts, let's now illustrate them with a concrete example: representing jazz musicians. Specifically, we're going to look at a **small corpus of 15 prominent jazz musicians** and excerpts of their biographies from Wikipedia. Here are excerpts from a few jazz musician biographies:

3. See "Example: Attribute Selection with Information Gain" on page 56.

Charlie Parker

Charles “Charlie” Parker, Jr., was an American jazz saxophonist and composer. Miles Davis once said, “You can tell the history of jazz in four words: Louis Armstrong. Charlie Parker.” Parker acquired the nickname “Yardbird” early in his career and the shortened form, “Bird,” which continued to be used for the rest of his life, inspired the titles of a number of Parker compositions, [...]

Duke Ellington

Edward Kennedy “Duke” Ellington was an American composer, pianist, and big-band leader. Ellington wrote over 1,000 compositions. In the opinion of Bob Blumenthal of *The Boston Globe*, “in the century since his birth, there has been no greater composer, American or otherwise, than Edward Kennedy Ellington.” A major figure in the history of jazz, Ellington’s music stretched into various other genres, including blues, gospel, film scores, popular, and classical.[...]

Miles Davis

Miles Dewey Davis III was an American jazz musician, trumpeter, bandleader, and composer. Widely considered one of the most influential musicians of the 20th century, Miles Davis was, with his musical groups, at the forefront of several major developments in jazz music, including bebop, cool jazz, hard bop, modal jazz, and jazz fusion.[...]

Even with this fairly small corpus of fifteen documents, the corpus and its vocabulary are too large to show here (nearly 2,000 features after stemming and stopword removal) so we can only illustrate with a sample. Consider the sample phrase “*Famous jazz saxophonist born in Kansas who played bebop and latin.*” We could imagine it being typed as a query to a search engine. How would it be represented? It is treated and processed just like a document, and goes through many of the same steps.

First, basic stemming is applied. Stemming methods are not perfect, and can produce terms like *kansa* and *famou* from “Kansas” and “famous.” Stemming perfection usually isn’t important as long as it’s consistent among all the documents. The result is shown in Figure 10-2.

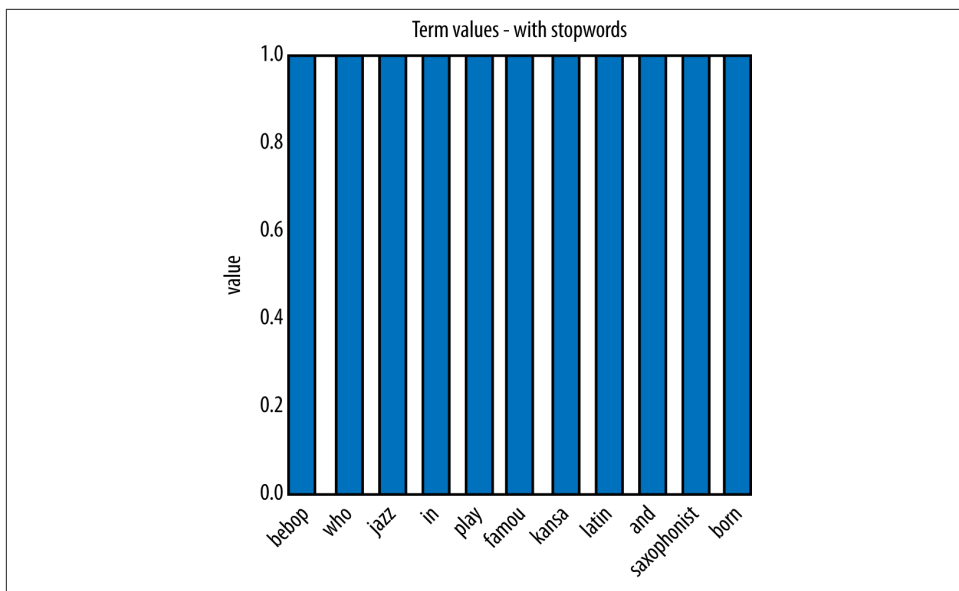


Figure 10-2. Representation of the query “Famous jazz saxophonist born in Kansas who played bebop and latin” after stemming.

Next, stopwords (in and and) are removed, and the words are normalized with respect to document length. The result is shown in Figure 10-3.

These values would typically be used as the Term Frequency (TF) feature values if we were to stop here. Instead, we’ll generate the full TFIDF representation by multiplying each term’s TF value by its IDF value. As we said, this boosts words that are rare.

Jazz and play are very frequent in this corpus of jazz musician biographies so they get no boost from IDF. They are almost stopwords in this corpus.

The terms with the highest TFIDF values (“latin,” “famous,” and “kansas”) are the rarest in this corpus so they end up with the highest weights among the terms in the query. Finally, the terms are renormalized, producing the final TFIDF weights shown in Figure 10-4. This is the feature vector representation of this sample “document” (the query).

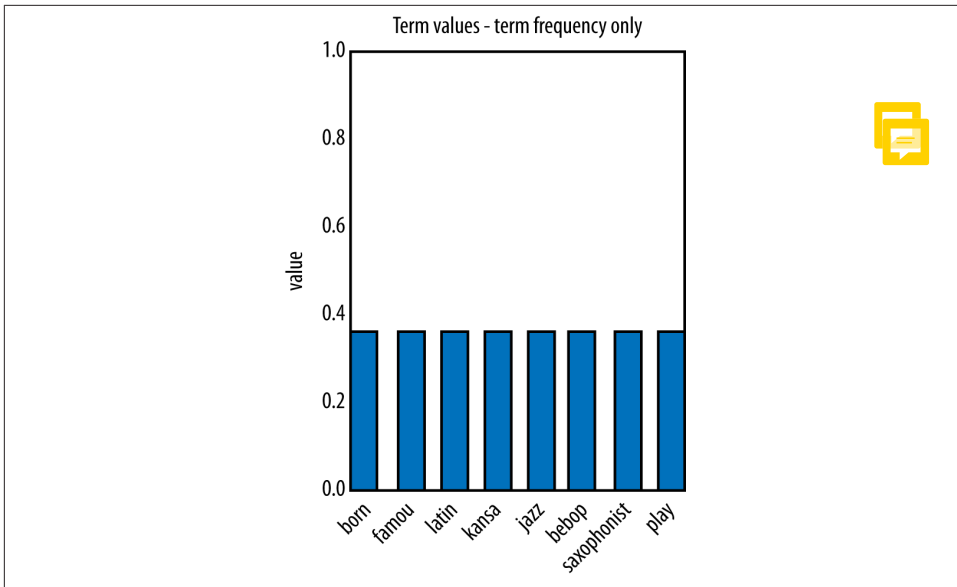


Figure 10-3. Representation of the query “Famous jazz saxophonist born in Kansas who played bebop and latin” after stopwords removal and term frequency normalization.

Having shown how this small “document” would be represented, let’s use it for something. Recall in [Chapter 6](#), we discussed doing nearest-neighbor retrievals by employing a distance metric, and we showed how similar whiskies could be retrieved. We can do the same thing here. Assume our sample phrase “Famous jazz saxophonist born in Kansas who played bebop and latin” was a search query typed by a user and we were implementing a simple search engine. How might it work? First, we would translate the query to its TFIDF representation, as shown graphically in [Figure 10-4](#). We’ve already computed TFIDF representations of each of our jazz musician biography documents. Now all we need to do is to compute the similarity of our query term to each musician’s biography and choose the closest one!

For doing this matching, we’ll use the Cosine Similarity function ([Equation 6-5](#)) discussed back in the starred section “[Other Distance Functions](#)” on [page 158](#). Cosine similarity is commonly used in text classification to measure the distance between documents.

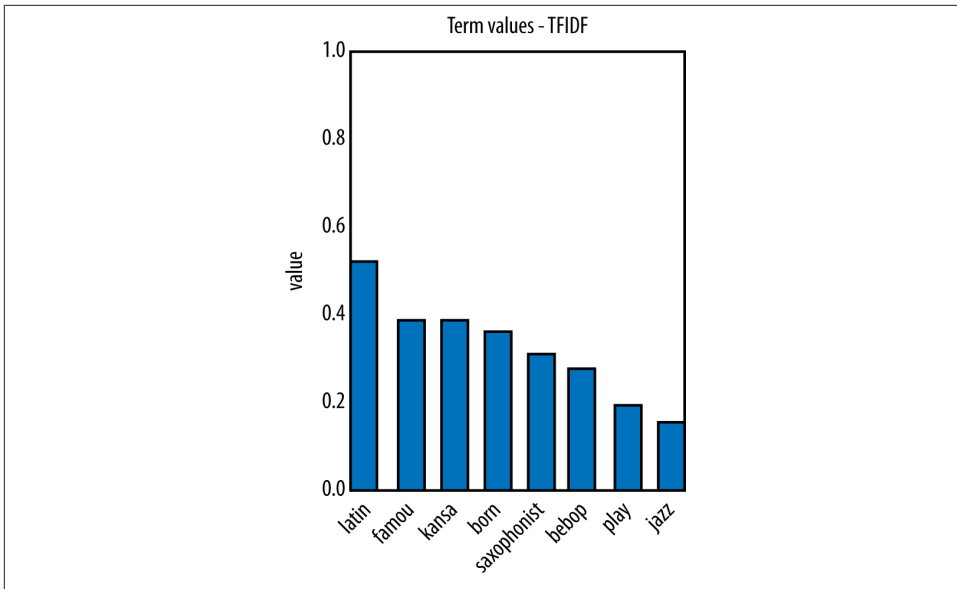


Figure 10-4. Final TFIDF representation of the **query** “Famous jazz saxophonist born in Kansas who played bebop and latin.”

Table 10-4. Similarity of each musician’s text to the query ‘Famous jazz saxophonist born in Kansas who played bebop and latin,’ ordered by decreasing similarity.

Musician	Similarity	Musician	Similarity
Charlie Parker	0.135	Count Basie	0.119
Dizzie Gillespie	0.086	John Coltrane	0.079
Art Tatum	0.050	Miles Davis	0.050
Clark Terry	0.047	Sun Ra	0.030
Dave Brubeck	0.027	Nina Simone	0.026
Thelonius Monk	0.025	Fats Waller	0.020
Charles Mingus	0.019	Duke Ellington	0.017
Benny Goodman	0.016	Louis Armstrong	0.012

As you can see, the closest matching document is Charlie Parker—who was, in fact, a saxophonist born in Kansas and who played the bebop style of jazz. He sometimes combined other genres, including Latin, a fact that is mentioned in his biography.